

武汉大学计算机学院

本科生课程设计报告

基于 SSA 的 RISC-V 优化编译器设计与实现

专 业 名 称 : 计算机科学与技术
课 程 名 称 : 程序语言理论与编译技术
指 导 教 师 : 袁梦霆 教授
学 生 1 : 2023302121009 刘鼎琨
学 生 2 : 2023302131042 谢巧
学 生 3 : 2023302111012 郑慧琳
学 生 4 : 2023301042114 柯昀志

二〇二五年七月

郑 重 声 明

本小组呈交的设计报告，是在指导老师的指导下，独立进行实验工作所取得的成果，所有数据、图片资料真实可靠。尽我所知，除文中已经注明引用的内容外，本设计报告不包含他人享有著作权的内容。对本设计报告做出贡献的其他个人和集体，均已在文中以明确的方式标明。本设计报告的知识产权归属于培养单位。

成员签名：	<u>刘晶晶</u>	日期：	<u>2025.8.13</u>
成员签名：	<u>郭慧琳</u>	日期：	<u>2025.8.13</u>
成员签名：	<u>谢巧</u>	日期：	<u>2025.8.13</u>
成员签名：	<u>柯晓志</u>	日期：	<u>2025.8.13</u>

摘要

本报告介绍了一个面向 RISC-V 架构、基于 SSA 优化的 ToyC 编译器的设计与实现。编译器采用三段式架构，前端实现了 ToyC 语言的词法与语法分析，中端构建了基于控制流图的 SSA 中间表示，并集成了常量传播、死代码消除、循环优化等多种优化遍。后端实现了图着色寄存器分配和 RISC-V 汇编代码生成，并通过窥孔优化提升代码质量。项目重点在于 SSA 构建与解构和模块化的优化框架。系统测试覆盖 ToyC 语言全部核心特性，验证了编译器的语义正确性和鲁棒性。

关键词：SSA 中间表示；RISC-V；编译器设计；优化遍；

目 录

1 实验目的和意义	1
1.1 实验目的	1
1.2 实验意义	1
2 系统总体设计	3
2.1 编译器总体架构	3
2.2 开发环境与相关技术	4
2.2.1 开发语言与工具	4
2.2.2 RISC-V 指令集架构	5
2.3 设计亮点	5
2.3.1 静态单赋值形式 (SSA) 中间表示	5
2.3.2 经典的图着色寄存器分配	5
3 系统详细设计与实现	7
3.1 前端设计与实现	7
3.1.1 前端架构设计	7
3.1.2 语法规则的调整	7
3.1.3 注释处理	8
3.2 中端设计与实现	8
3.2.1 静态单赋值中间表示设计	8
3.2.2 自定义 IR 与控制流图的构建	9
3.2.3 优化遍#1: 循环优化	10
3.2.4 优化遍#2: 常量传播与折叠	10
3.2.5 优化遍#3: 不可达代码与死代码消除	11
3.2.6 SSA 解构与 Phi 结点消除	11
3.3 后端设计与实现	12
3.3.1 寄存器分配: 基于图着色的实现	12
3.3.2 RISC-V 指令生成	14
3.3.3 窥孔优化	15

4 系统测试与结果分析	1 6
4.1 测试方案设计.....	1 6
4.1.1 测试框架设计	1 6
4.1.2 测试集设计与关键指标	1 6
4.2 功能性测试与验证.....	1 6
4.3 优化效果对比分析.....	1 7
5 总结与展望.....	1 9
5.1 工作总结.....	1 9
5.2 不足与展望.....	2 0
6 小组成员和分工.....	2 1
致 谢	2 3
参考文献	2 4

1 实验目的和意义

1.1 实验目的

本次编译原理课程实验的核心目标，是完整地设计、实现并验证一个面向 RISC-V 架构的优化编译器，从而将编译原理的理论知识体系化、工程化。

在前端层面，实验旨在掌握核心的语法解析技术。我们基于形式文法，利用 OCaml、ocamllex 及 ocamllyacc 工具链，为一种类 C 语言子集构建一个健壮的词法与语法分析器，并生成其抽象语法树（AST），为后续所有阶段提供规范化的结构输入。进而，实验的目标聚焦于攻克中端优化的核心难题。我们设计了一套自定义的中间表示（IR）与控制流图（CFG），并在此之上，选择构建一个基于静态单赋值（SSA）形式的模块化优化框架。该框架的实现工作不仅涵盖了常量传播、死代码消除等多个具体优化遍，更深入到了 SSA 构建与解构的核心技术挑战，即 Φ 函数的正确插入与消除。在后端层面，我们的目标是实现经典的后端代码生成算法，将优化后的 IR 映射到真实的 RISC-V 指令集。此阶段的关键目标是完整实现经典的图着色寄存器分配算法，包括其前置的活跃性分析、干涉图构建以及溢出处理逻辑，以期生成功能正确且可在模拟器上运行的高质量汇编代码。

1.2 实验意义

从理论角度看，本项目的意义在于将抽象的理论知识具象化为可运行的工程实体。通过亲手实现，我们对编译理论的理解不再停留在概念层面。例如，在实现活跃性分析时，我们深刻体会了数据流分析中“不动点”思想的迭代过程；在处理 SSA 形式时，我们真正理解了 Φ 函数是如何将复杂的“定义-使用链”显式化，从而极大地简化后续优化算法；而在实现图着色算法时，我们将图论知识与编译器后端这一经典场景紧密结合，理解了理论算法在面对工程约束时的权衡与挑战。

从工程实践的角度看，其意义体现在对复杂软件系统工程能力的全面锻炼。一方面，我们成功构建了一个从前端到后端、包含多个复杂模块的端到端系统，这锻炼了我们在接口设计、模块解耦和流程集成方面的宏观设计能力。另一方面，与许多仅实现基础功能的项目不同，我们着重挑战并实现了 SSA 和图着色寄存器分配这两个技术难点。调试和验证这些复杂算法的过程，极大地提升了我们解

决深度技术问题的能力。此外，选择开源的 RISC-V 作为目标架构，并使用 Spike 模拟器进行验证，使我们对现代计算机体系结构与编译技术的协同工作模式有了切实的认识，完成了从理论到可验证结果的闭环。

2 系统总体设计

2.1 编译器总体架构

本编译器项目严格遵循现代编译器设计“三段式”（Three-Phase）的设计原则，将整个编译过程解耦为前端（Frontend）、中端（Middle-end）和后端（Backend）三个相对独立的部分，其核心优势在于实现了高度的模块化，使得各阶段职责明确、接口清晰，便于独立开发与维护。同时，这种设计保证了中端优化的平台无关性，提高了编译器的可移植性与复用性。通过将前端的源码解析与后端的机器代码生成进行关注点分离，有效简化了编译器的整体复杂度。

本编译器的总体架构如图 2.1 所示。它宏观地展示了编译器三个核心组成部分以及它们之间的数据流。

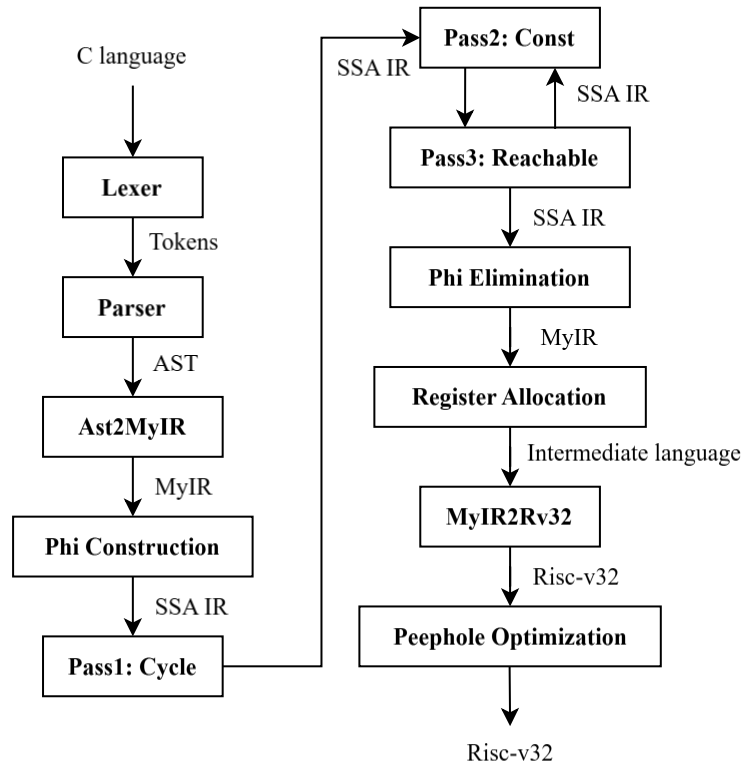


图 2.1: 编译器架构流程图

基于此三段式架构，本编译器的具体执行流程是一个由多个阶段构成的精确流水线，它将输入的 C 语言子集源代码逐步分析、转换并最终生成目标汇编代码。下面将详细阐述这一流程。

编译流程始于前端。当前端模块接收到源代码文件后，首先由 `lexer.mll` 实现

的词法分析器将字符流分解为词法单元（Token）序列。随后，此序列被传递给由 `parser.mly` 和 `ast.ml` 共同定义的语法分析器。语法分析器依据语言语法规则进行解析，最终为源程序中的每个函数构建一棵抽象语法树（AST）。AST 作为一种结构化的中间形式，忠实地反映了代码的层次结构，并作为前端的最终输出，被完整地移交给中端处理。

进入中端后，首要任务是将 AST 转换为一种更利于分析和优化的自定义中间表示（IR）。`ast2ir.ml` 模块负责遍历 AST，并生成基于控制流图（CFG）的 IR。为实现更高效的分析与代码变换，此 IR 紧接着被 `ssa_builder.ml` 模块转换为静态单赋值（SSA）形式。在此基础上，一系列独立的优化遍（Passes）被顺序执行，包括常量传播与折叠、不可达代码消除以及死代码消除等。所有优化完成后，`ssa_destruction.ml` 模块负责将代码移出 SSA 形式，主要是通过消除 Φ 函数来恢复常规的变量赋值形式。经过多轮优化的 IR，是整个中端处理阶段的产物。

最后，流程进入后端。后端接收中端优化后的 IR，并将其映射到目标 RISC-V 架构。这一过程始于活跃性分析，为后续的寄存器分配提供关键的数据支持。接着，核心的寄存器分配阶段采用经典的图着色算法，将无限的虚拟寄存器映射到有限的物理寄存器上。完成分配后，`ir2rv.ml` 模块负责进行指令选择与生成，将 IR 指令翻译成最终的 RISC-V 汇编指令序列。作为最后一道工序，`peephole.ml` 模块会对生成的汇编代码进行窥孔优化，以消除一些局部的冗余。整个流程的终点，是将优化后的汇行代码写入到 .s 文件中，完成编译任务。

2.2 开发环境与相关技术

2.2.1 开发语言与工具

本编译器采用 OCaml 作为核心开发语言。OCaml 强大的静态类型系统与模式匹配机制，为处理编译器中普遍存在的树状及代数数据结构（如抽象语法树与中间表示）提供了极大的便利与安全性。前端的构建依托于 OCaml 生态的标准工具链：`Ocamllex` 用于生成词法分析器，`Ocamlyacc` 用于生成语法分析器。整个项目的编译、依赖管理及模块化构建则由 `Dune` 工具统一负责。为验证编译结果的正确性，我们采用 RISC-V GCC 交叉编译器和 `Spike` 模拟器搭建了自动化测试环境。

2.2.2 RISC-V 指令集架构

本编译器选择 RISC-V 作为目标硬件平台。RISC-V 是一个开源、精简的现代指令集架构，其指令格式规整、规范清晰，显著降低了后端代码生成的实现复杂度。

我们的实现并未覆盖完整的 RISC-V 规范，而是针对性地构建了一个满足 C 语言子集编译需求的 64 位基础整数指令子集（RV64I）。根据 `riscv_instr.ml` 的定义，该子集的功能覆盖了程序运行所需的核心操作。在整数运算与逻辑判断方面，实现了包括 `add`, `sub`, `mul` 等寄存器-寄存器算术指令，`addi` 等立即数算术指令，以及 `sgt`, `slt`, `seqz`, `snez` 等一系列比较指令。内存访问则通过核心的 `lw`(load word) 和 `sw` (store word) 指令完成，以支持栈操作与数据读写。程序的控制流由无条件跳转 `j`、条件分支 `bnez`、函数调用 `call` 及返回 `ret` 指令共同构建。此外，为支持代码生成与链接，我们还定义了 `li`, `mv` 等常用伪指令，并集成了 `.globl` 和 `.text` 等必要的汇编器指令。

2.3 设计亮点

2.3.1 静态单赋值形式（SSA）中间表示

本项目中端设计的核心是其 IR 完全基于静态单赋值（SSA）形式。SSA 是现代优化编译器的基石，它通过保证每个变量仅被赋值一次，并将不同控制流路径的变量值通过 Φ 函数在交汇点合并，从而将复杂的迭代数据流分析问题转化为结构更清晰、更易处理的图论问题。这一特性极大地简化了如常量传播等多种优化算法的实现，同时提升了分析的精度和效率。

我们的 SSA 构建方案参考了 Cytron 算法的核心思想，结合控制流图（CFG）和数据流分析，设计数据结构管理变量版本和 Phi 节点插入。具体做法是：在遍历 AST 和生成 IR 时，遇到变量定义即分配新 SSA 变量，分支和循环合流点根据前驱块的变量状态插入 Phi 节点，保证 SSA 单赋值语义。虽未显式计算支配树和支配边界，但整体流程实现了 SSA 的主要功能，为后续优化提供了结构清晰且高度可分析的基础。

2.3.2 经典的图着色寄存器分配

在后端，我们实现了经典的基于图着色的全局寄存器分配算法，这是生成高性能目标代码的关键。与简单的局部算法不同，图着色从整个函数的全局视角出发，旨在将程序中的活跃变量尽可能地分配到有限的物理寄存器中，从而最大化地减少因寄存器不足而导致的低效内存读写（**Spill**），直接提升最终代码的运行性能。

我们通过精确的活跃性分析，以计算出程序中所有变量的生命周期。分析结果随即被用于构建干涉图，图中节点代表变量，边则表示两个变量的生命周期存在重叠，因而不能共用同一寄存器。在此图的基础上，核心的启发式着色算法被执行，同时我们还集成了合并（**Coalescing**）优化，以主动消除冗余的 `mov` 指令。最后，为了应对物理寄存器不足的场景，我们设计了完善的溢出（**Spilling**）处理机制，基于代价模型来选择溢出代价最低的变量，确保在寄存器压力大的情况下性能损失也能被降到最低。对这一经典且复杂的寄存器分配策略的完整实现，是本项目在后端优化上的一项核心工作，也是保证最终生成代码运行效率的关键环节之一。

3 系统详细设计与实现

3.1 前端设计与实现

3.1.1 前端架构设计

本项目遵循编译器前端的经典架构，主要由词法分析器（**Lexer**）和语法分析器（**Parser**）两部分组成。词法分析器负责将源代码分解为一系列的词法单元（**Tokens**），而语法分析器则在此基础上，根据预定义的文法规则，将词法单元流构建成为抽象语法树（**AST**）。这种模块化的设计使得各阶段职责清晰，便于开发、测试与维护。**OCamllex** 工具用于生成词法分析器，通过正则表达式定义词法规则，将输入字符流转换为词法单元。**OCamlyacc** 工具则用于生成语法分析器，基于 **LALR(1)** 文法定义语法规则，将词法单元流解析为 **AST** 结构。最终生成的 **AST** 作为后续编译器阶段（如语义分析、优化和代码生成）的输入，为程序的进一步处理提供了统一且结构化的表示。

3.1.2 文法规则的调整

在前端转换器的实现过程中，原始文法规则与 **OCamlyacc** 工具的兼容性问题导致了必要的调整。具体而言，原始文法中 **CompUnit -> FuncDef+** 这一规则使用了正则表达式中的 **+**（表示一个或多个 **FuncDef**），而 **OCamlyacc**（以及大多数 **LALR(1)** 解析器生成器）通常不支持在 **BNF** 规则中直接使用此类正则表达式操作符。为了解决这一问题，我们采用了标准 **BNF** 范式进行等价转换。原始的 **FuncDef+** 被重写为两个相互递归的规则：

```
compunit:
funcdef compunit_tail { $1 :: $2 }

compunit_tail:
funcdef compunit_tail { $1 :: $2 }
| { [] }
```

这种转换方式确保了 **CompUnit** 至少包含一个 **FuncDef**，并且可以包含任意数量的后续 **FuncDef**，从而精确地表达了原始文法中“一个或多个”的语义，同时满足了 **OCamlyacc** 的输入要求。这种模式在解析器设计中非常常见，用于处理重复出现的语法结构，例如语句列表、参数列表等。通过这种方式，我们成功地将抽象的文法概念映射到具体的解析器实现中，保证了文法的正确性和解析

的有效性。

3.1.3 注释处理

在词法分析阶段，对源代码中的注释进行识别和消除是前端处理的重要一环，这确保了后续语法分析器只处理有效的程序代码。尽管提供的代码中未包含注释处理逻辑，但可以通过在 `ocamllex` 规则中添加相应的正则表达式来实现。对于类 C 语言，常见的注释形式有两种：单行注释（`// ...`）和多行注释（`/* ... */`）。为了在 `lexer.mll` 中实现注释的匹配和消除，添加以下规则：

```
rule read_token = parse
| whitespace+ { read_token lexbuf }
| "//" [^
]* { read_token lexbuf } (* 匹配并忽略单行注释 *)
| "/*" ([^*] | '*' [^/])* "*" { read_token lexbuf } (* 匹配并忽略多行注释 *)
(* ... 其他词法规则 ... *)
```

其中，`"//" [^`]* 用于匹配单行注释，它从 `//` 开始，一直匹配到行尾（不包括换行符）。`"/*" ([^*] | '*' [^/])* "*"` 用于匹配多行注释，它从 `/*` 开始，匹配任意非 `*` 字符，或者 `*` 后面不是 `/` 的字符，直到遇到 `*/`。这两种规则都将匹配到的注释内容丢弃，并通过递归调用 `read_token lexbuf` 来继续读取下一个有效的词法单元，从而实现了注释的有效消除，保证了语法分析器接收到的词法单元流是纯净的。

3.2 中端设计与实现

3.2.1 静态单赋值中间表示设计

单一赋值原则（SSA）是一种在编译器设计中用于处理变量赋值和控制流的技术，它要求每个变量在程序中只能被赋值一次。为了实现这一点，编译器会为每次变量赋值生成新的临时变量，从而确保每个变量都有一个唯一的定义。我们通过定义一系列的数据结构去保证在 SSA 形式中每个变量都有明确且唯一的定义，如 `current_def` 用于维护每个变量的当前 SSA 定义，而 `back_edges` 和 `break_edges` 则分别用于存储循环回边和 `break` 语句的变量定义状态，`loop_scope_stack` 则帮助处理嵌套循环中的变量遮蔽问题。为了确保每个变量的

定义是唯一的，当一个变量被赋值时，编译器会为它创建一个新的临时变量并将其加入到 `current_def` 中。这意味着每次对变量进行定义时，都会生成一个新的 SSA 值。如果变量尚未在当前定义中找到，则会通过生成 `Load` 指令从内存加载其当前值，并将其赋给一个临时变量，以确保 SSA 格式下的单一赋值规则得以遵守。

处理控制流的分支结构时，SSA 会插入 `Phi` 节点来解决控制流汇合点的变量定义合并问题。在每个分支的开始，编译器会恢复分支前的变量定义状态，并在汇合点插入 `Phi` 节点，这些 `Phi` 节点记录了每个前驱块及其对应的操作数，从而正确地合并不同路径上的变量值。

对于循环控制流的处理，编译器会在循环头部创建一个不完整的 `Phi` 节点，该节点只有前驱边信息。在遍历回边时，编译器会通过 `back_edges` 哈希表收集回边的变量定义，并且通过 `loop_scope_stack` 来处理循环内部的变量遮蔽问题。循环结束时，回边的信息将被合并到 `Phi` 节点中，以确保循环内外的变量定义能够正确处理。

在变量遮蔽的处理上，编译器会在 `loop_scope_stack` 中跟踪不同作用域中的变量定义，以检测哪些变量被遮蔽。当发现某个外部变量在内层作用域中被遮蔽时，编译器会从原始内存地址加载该变量的值，并通过 `Load` 指令确保在 SSA 中正确地使用该变量。

3.2.2 自定义 IR 与控制流图的构建

为实现基本功能，编译器还需要定义一套简化版的中间代码（IR）。模仿 LLVM 的中间代码，IR 中的指令涵盖了控制流、计算、内存访问等方面。例如，控制流指令包括 `ret`、`br` 和 `jump`，而计算指令包括二元运算指令（`binop`）、整数比较指令（`icmp`）等。此外，还包括内存访问指令如 `alloca`、`store`、`load` 等，支持对数据的管理和访问。函数调用指令 `call` 则允许执行外部函数并可选地返回结果。

为了构建控制流图（CFG），每个基本块需要定义唯一的标签，并记录它内部的指令以及前驱和后继关系。基本块中的指令按照执行顺序排列，`defs` 字段记录该基本块中定义的所有变量。控制流图的每个节点包含指令列表、前驱和后继信息，以及用于活跃变量分析的 `def`、`use`、`live_in` 和 `live_out` 集合。

控制流图的构建依赖于指令的类型。例如，`jump` 指令会创建一条无条件跳转

边，`br` 指令则会创建两条条件跳转边，分别对应真分支和假分支。`ret` 指令则不创建任何跳转边，因为它表示函数的退出。通过这些控制流指令和相应的图结构，编译器能够动态地构建和修改控制流图，以支持优化和其他编译过程。

3.2.3 优化遍#1：循环优化

在程序执行过程中，循环通常占据了主要的计算时间，因此针对循环结构的优化对提升整体性能至关重要。作为第一阶段，这部分的优化目标是识别并优化程序中的循环结构，我们通过以下几种方式改善循环体执行效率：移除循环中不变的计算、消除循环内不必要的内存操作，以及适度展开小规模循环以降低分支开销。

循环识别的核心方法基于控制流分析。首先本部分通过深度优先遍历程序流程，当发现当前块的后继指向先前访问过的块时，即判定此处为循环的起点。识别循环后，我们会分析循环头的 `Phi` 节点，自动寻找归纳变量。对于每次以固定步长递增的变量，我们在循环体内检测到其乘法运算后，直接将其转换为加法操作，实现强度削减。例如， $x * 2$ 会被替换为 $x + x$ ，减少算术指令的执行成本。此外，我们还会消除冗余的内存操作。通过分析连续的 `Store` 指令，只保留最后一次写入，去除死存储，提升内存访问效率。这个过程会递归检查每个循环块的指令序列，自动移除无效的存储操作。

整个循环优化过程依赖 `SSA` 结构带来的数据流清晰和变量版本唯一性，使归纳变量识别、算术简化和死存储消除都能高效且安全地实现。我们在实现时始终优先保证代码的正确性和健壮性，面对复杂循环保持分析和匹配的保守性。

3.2.4 优化遍#2：常量传播与折叠

程序的运行过程存在许多在编译时即可简化的计算过程，常量优化的目标就是将这些可预知的信息提前处理，消除运行时的冗余操作。该优化在 `SSA` 形式的帮助下效果尤其良好，因为变量的单一定义点消除了值变化的不确定性，使常量状态跟踪变得可靠，效率也相对更高。本部分的优化可以概括为三种作用：直接替换已知常量值、预先计算常量表达式结果、以及简化基于常量判断的控制流程，共从而进一步降低计算开销和控制流复杂度。

这项技术的基础是逐步传播已知信息的数据流分析。本部分采用的方式是在 `SSA` 架构下维护记录变量状态的全局映射表，因为每个变量版本在程序中只有一次定义，可以通过多次遍历程序指令直至状态稳定。处理每条指令时，利用 `SSA`

的可见赋值链快速确定值来源；在合并点 PHI 节点处，由于输入路径相对明确，可以在安全的前提下判断常量一致性。而本部分针对关键的特殊结构也设计了特殊的处理机制：当所有输入路径传递相同常量时，直接替换为常量赋值；当仅剩单一有效路径时，利用 SSA 而变得清晰的赋值链，也就能退化为简单的赋值指令。

这部分的核心挑战在于值依赖链的精确追踪与安全性保障之间的平衡。在 SSA 形式下，状态依赖表现为 PHI 节点输入的明确传递关系，更容易解析依赖。安全性保障方面，通过设置合理的最大迭代次数来进行约束。

3.2.5 优化遍#3：不可达代码与死代码消除

随着程序迭代开发，代码中常常会积累一些无效或冗余的部分。本阶段的目标是精简代码体积，优化内存访问模式。借助 SSA 形式提供的精确数据流信息，死代码检测和消除变得更高效可靠。我们主要分为两个阶段：首先移除所有不可达的基本块，然后清除对程序输出无影响的冗余指令，尤其关注内存操作的优化。SSA 特性降低了分析复杂度，使活跃变量识别更准确。

具体实现中，我们结合 SSA 的数据流和控制流信息，首先从程序入口出发标记所有可达的基本块，清理掉不会被执行到的代码区域。随后，利用 SSA 的明确定义关系，从关键操作（如函数调用、返回、分支等）逆向标记所有直接影响程序行为的指令，识别并移除无用代码。针对内存操作，我们设计了专门的分析策略，利用 SSA 的点对点赋值结构，识别并消除冗余的存储指令，提升内存访问效率。

在安全性方面，我们对未知函数调用和全局状态采取保守策略，确保不会误删有副作用的代码。SSA 框架有助于限定优化影响范围，尤其在循环体内的内存优化上，能够识别重复存储模式，使分析更模块化，优化过程既安全又有效。

3.2.6 SSA 解构与 Phi 结点消除

前述章节中，我们充分利用 SSA 形式的结构优势，完成了包括常量传播与折叠、不可达代码与死代码消除在内的一系列关键优化。 Φ 函数本身在任何物理硬件上都没有直接对应的指令。因此，必须将这种“分析形态”的 IR，转换回一种能够直接映射到线性机器指令的“执行形态”。

此过程的主要挑战在于如何正确地将 Φ 函数的抽象语义，等价地转换为一系列具体的 mov 指令。一个看似直接的方法是，对于形如 $y = \Phi((x1, L1), (x2, L2))$

的指令，分别在其每个前驱基本块（L1, L2）的末尾插入相应的 `mov` 指令（如 `mov y, x1`）。然而，当一个前驱块拥有多个后继，而后继块也拥有多个前驱时，连接它们的控制流边被称为“关键边”（Critical Edge）。在关键边上，这种简单的插入策略会失效，因为它会导致 `mov` 指令在错误的执行路径上被执行，从而破坏程序的原始逻辑。

为了稳健地解决这一问题，我们采用了一个基于边分裂（Edge Splitting）策略的 SSA 解构算法，将解构过程严谨地划分为四个阶段。首先，遍历 CFG，识别所有 Φ 函数，并将其所需的拷贝操作与相应的 CFG 边关联起来。其次，在插入任何代码之前，检查所有需要承载拷贝操作的边。如果某条边是关键边，则动态地创建一个只包含一条无条件跳转的新基本块，插入到这条边的中间，从而将其“分裂”为两条非关键边。所有边都变得安全之后，第三阶段才会执行真正的拷贝指令插入。为避免多个并行的 Φ 节点赋值（如 $x=\Phi(y, \dots)$ 和 $y=\Phi(x, \dots)$ ）引发的逻辑错误，我们通过引入临时变量来保证并行拷贝语义的正确实现。最后，当所有 Φ 函数的语义都被等价的 `mov` 指令序列替代后，算法会遍历所有基本块，将这些逻辑上已为空的 Φ 指令彻底移除。

经过 SSA 解构遍的处理，中间表示被完全转换回一种线性的、无 Φ 函数的、更接近物理机模型的指令序列。这种形式的 IR 已经消除了所有分析阶段的抽象构造，可以直接用于后续的活跃性分析、寄存器分配以及最终的目标代码生成，为编译器的后端阶段做好了充分的准备。

3.3 后端设计与实现

3.3.1 寄存器分配：基于图着色的实现

将中间表示中无限的虚拟寄存器，映射到目标硬件有限的物理寄存器上，是后端代码生成面临的核心矛盾。为应对这一挑战，我们完整地实现了经典的基于图着色的全局寄存器分配算法，它将一个看似棘手的资源调度问题，巧妙地重塑（recast）为一个定义清晰、拥有成熟求解思路的图论问题，从而能在整个函数的范围内，系统性地寻求最优的寄存器分配方案。

实现寄存器分配必须先进行活跃性分析（Liveness Analysis）。我们采用经典的反向数据流分析过程，为 CFG 中的每一个基本块计算出入口处的活跃变量集

live_in 和出口处的活跃变量 live_out。我们的实现通过不动点迭代来求解这组方程。为加速收敛，迭代严格按照基本块的反向后序（Reverse Postorder）进行，直至所有基本块的活跃变量集不再发生变化。此过程产出的每个程序点的精确活跃变量集，是构建后续冲突关系的唯一依据。

在获得精确的活跃性信息后，我们构建了干涉图（Interference Graph）。图中每个节点代表一个需要被分配物理寄存器的虚拟寄存器（或一个预着色的物理寄存器）。如果两个变量的生命周期存在重叠，我们就在代表它们的节点之间连接一条边，表示它们不能被分配到同一个物理寄存器。对于一个被定义的变量 d ，在它与该指令出口处的所有活跃变量之间都建立干涉关系。特别地，对于 `mov x, y` 这样的传送指令，我们记录下 (x, y) 的传送关系，但暂时不在它们之间添加干涉边，这为后续的合并优化提供了便利；对于函数调用指令，所有在调用期间活跃的变量，都会与所有调用者保存（caller-saved）的物理寄存器产生干涉，以确保跨函数调用的数据正确性。

干涉图构建后，进行图着色分配寄存器。由于图着色是 NP 完全问题，我们采用启发式算法，基于 Briggs 和 George 的思想，迭代地简化图来寻找一个可行的 K -着色方案（ K 为可用物理寄存器的数量）。核心是维护一系列工作表（worklist），通过精心设计的优先级循环来处理图中的节点。算法会优先尝试简化（Simplify）图，通过反复移除图中度数小于 K 的节点（总能被成功着色的节点），将它们压入一个栈中。同时，积极地尝试合并（Coalesce）由 `mov` 指令连接的两个节点，若合并成功，不仅能消除一条冗余的 `mov` 指令，还能减少图的规模。我们采用了保守的合并策略，只有当合并后的新节点度数依然小于 K 时才执行，以避免产生一个难以着色的高密度图。当一个与 `mov` 指令相关的低度数节点无法被安全合并时，将其冻结（Freeze），放弃对该 `mov` 的合并尝试，并把该节点视为一个可简化的普通节点。最终，当工作表中只剩下度数大于等于 K 的高冲突节点，无法进行任何简化、合并或冻结时，必须选择一个节点进行溢出（Spill）。我们基于节点的度数选择一个冲突最严重的节点，将其标记为“待溢出”，然后也像简化节点一样将其从图中移除并压入栈中。

图中所有节点都被移除后，开始着色。从栈中依次弹出节点，将其重新加入图，并为它选择一个与所有已着色邻居不同的颜色。如果一个节点在此时能够找到颜色，则分配成功。然而，对于被标记为“待溢出”的节点，在此时大概率无法

找到可用颜色，从而成为“真正的溢出”。我们的实现包含了一个完善的溢出处理与程序重写（**Spilling and Code Rewriting**）机制。当一个变量被确认溢出后，编译器会在当前函数的栈帧中为该溢出变量预留一个专用的内存槽位。随后，编译器会遍历整个函数的 IR，在每个定义该变量的指令之后，自动插入一条 **store** 指令将其值存入栈中；在每个使用该变量的指令之前，自动插入一条 **load** 指令将其值从栈中读回一个新的临时虚拟寄存器。当 IR 重写完成后，溢出变量便从代码中消失了，取而代之的是一系列生命周期极短的临时变量。此时，整个寄存器分配流程将基于新的 IR 从头开始，重新进行活跃性分析、构建干涉图和新一轮的着色尝试。“尝试分配-溢出-重写-重启”的迭代循环，从理论上保证了分配算法的最终收敛。每一次循环，至少有一个冲突最严重的变量被移出寄存器分配的行列，从而降低了干 SAP 图的整体“压力”，直至所有变量都能被成功分配到一个位置。这一套完整的实现，确保了我们的编译器能够生成高质量且功能正确的汇编代码。

3.3.2 RISC-V 指令生成

寄存器分配完成后，编译器后端进入指令生成阶段。此时的中间表示（IR）已“物理化”：所有虚拟寄存器均已映射为具体的物理寄存器或栈帧中的偏移量。指令生成的核心任务，是将这种 IR 系统性地翻译为符合 RISC-V ABI 规范、结构严谨的汇编代码。

首先，对整个函数进行资源审计（**Resource Audit**），精确计算所需栈帧大小。遍历所有指令，判断是否存在 **Call** 指令以确定是否为非叶函数，决定是否需要保存返回地址 **ra**。分析 **def-use** 链和寄存器分配结果，统计实际用到的 **s** 系列被调用者保存寄存器，并为其分配合适空间。收集所有 **alloca** 定义的本地变量及溢出变量，为其分配栈空间。所有空间需求最终按 16 字节对齐，计算出栈帧总大小。

在函数入口处，生成标准前言（**Prologue**）：通过 **addi sp, sp, -framesize** 分配栈帧，依次用 **sw** 指令保存 **ra** 和所有用到的 **s** 系列寄存器。函数返回路径末尾，生成后语（**Epilogue**）：用 **lw** 指令恢复所有寄存器，收缩栈帧，最后执行 **ret**。

函数体翻译前，先对 CFG 进行线性化（**block scheduling**），确定基本块排放顺序，最大化利用 **fall-through** 特性。随后，逐块逐条将 IR 指令翻译为 RISC-V 指令。算术逻辑指令的操作数如为立即数，通过 **get_operand_reg** 加载到临时寄

寄存器参与运算。内存访问指令的地址由 `sp` 和预先计算的偏移量组成。`Br`（条件分支）指令结合线性化布局优化，若目标块紧邻当前块，则可省略跳转指令或通过条件反转实现 `fall-through`，生成更紧凑高效的分支代码。`Call` 指令严格遵循 ABI，参数置入 `a0-a7`，超过 8 个参数通过栈传递，返回值从 `a0` 获取。

最终，函数的前言、主体、后语被翻译为 RISC-V 指令列表，序列化为文本，并与 `.globl`、`.text` 等伪指令组合，写入 `.s` 文件，形成完整的编译单元。

3.3.3 窥孔优化

在经过前端解析、中端优化和后端指令选择等一系列自动化流程后，编译器生成的汇编代码在逻辑上已完全正确，但从局部看，仍可能存在一些由通用翻译规则所导致的、可预测的低效模式。为此，我们引入了窥孔优化（`Peephole Optimization`），通过一个极小的滑动窗口（“窥孔”）来扫描最终的 RISC-V 指令对象列表，一旦发现已知的低效模式，便将其替换为等价但更高效的指令序列。为确保优化效果的最大化，此过程会迭代执行，直至某次遍历不再产生任何变化，从而能够处理因前一次优化而新产生的优化机会。

我们的窥孔优化器能够识别并处理多种关键模式，这些模式按其优化目标可分为几类。第一类是消除冗余的寄存器间传送。例如，对于 `add t3, s0, s1` 后紧跟 `mv a0, t3` 这样的序列，优化器能识别出其意图是将运算结果直接存入 `a0`，因此会将其合并为单条指令 `add a0, s0, s1`，不仅消除了一条 `mov` 指令，也减少了对临时寄存器 `t3` 的依赖。同样，对于 `mv s0, s0` 这类无意义的自我传送指令，也会被直接移除。

第二类优化旨在利用指令集的特性进行强度削减。例如，当优化器看到 `li t0, 10` 后紧跟 `add a0, s0, t0` 时，它会理解到这可以被替换为单条的 `addi a0, s0, 10`，从而用一条指令完成了两者的功能。

最后一类优化专注于化简局部的控制流。在 CFG 线性化后，可能会产生 `j L1` 后紧邻 `L1: 标签` 的冗余跳转，优化器会将其安全地移除。更重要的是，我们的实现能够优化分支判断链。例如，当 `snez t0, s0` 后紧跟 `bnez t0, L1` 时，优化器能洞察到这整个序列的语义与单条 `bnez s0, L1` 指令完全等价，从而用一条指令替换了两条，提升分支判断的效率。

窥孔优化的累积效应是显著的，它有效地减少了最终代码的体积，通过降低总指令数和消除寄存器依赖来提升程序的运行效率。

4 系统测试与结果分析

4.1 测试方案设计

4.1.1 测试框架设计

为系统性、可重复验证编译器质量，我们设计并实现了一套综合性自动化测试框架。该框架的设计遵循三大核心原则：基于对比的验证、双维度评估与流程自动化。我们选择业界公认的 GCC 编译器并开启二级优化（-O2）作为“黄金参照物”，所有功能与性能指标均与此参照物进行直接、量化对比。测试框架同时覆盖功能正确性与优化有效性两个维度，由核心脚本驱动，实现一键式回归测试。

4.1.2 测试集设计与关键指标

为保证测试的广度与深度，我们设计了一套分类明确的测试用例集，并定义了多维度的关键性能指标用于最终评估。

测试用例集分为两大类别。第一类其核心目标是保证对 ToyC 语言特性的广度覆盖。该类别下设多个子类：`basic_tests` 覆盖基础算术与函数调用；`if_tests` 与 `while_tests` 分别专注于验证条件分支与循环控制流，包括对 `break` 和 `continue` 等复杂情况的测试；`comprehensive_tests` 与 `challenging_tests` 则提供语法结构高度交织的复杂场景，用以考核编译器在集成压力下的基础功能与鲁棒性。

第二类测试用例为 `optimization_tests`，旨在提供深度测试，以隔离并检验特定优化遍的有效性。例如 `constant_propagation` 用例包含大量可在编译期计算的常量表达式，而 `dead_code_elimination` 用例则包含不影响程序最终输出的冗余计算。为从不同维度量化编译器生成代码的质量。

我们采用总运行时间（Execution Time）作为性能指标。我们通过高精度计时工具，测量程序在 Spike 模拟器上完整运行所需的时间。为消除系统抖动带来的随机误差，性能测试模式会对每个用例迭代运行多次，并记录其总耗时。

4.2 功能性测试与验证

功能性测试全面覆盖 ToyC 语言的所有核心语法和语义特性。测试用例设计遵循“语法全覆盖、边界全覆盖、组合场景全覆盖”的原则，分为基础算术、条件分支、循环控制流、综合场景和挑战用例五大类。所有测试均通过自动化脚本批量运行，结果可复现，便于持续集成和回归验证。

如图 4.2.1 所示，所有功能测试用例均 100%通过，充分验证了编译器的语义正确性和鲁棒性。典型用例包括多层嵌套、复杂边界条件、break/continue 等混合控制流，均能正确处理。测试还覆盖了非法输入、边界溢出等异常场景，确保编译器具备良好的错误处理能力。

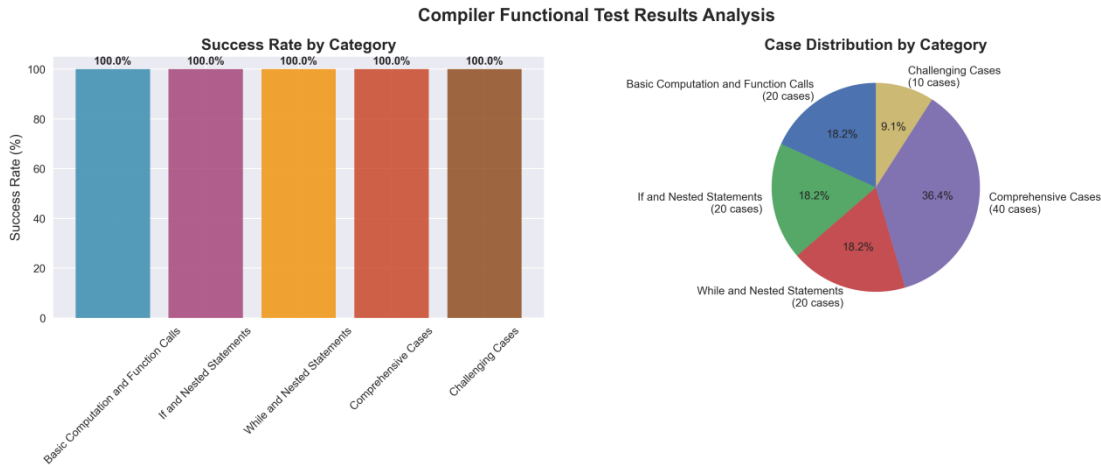


图 4.2.1 功能测试结果图

4.3 优化效果对比分析

优化效果测试主要针对常量传播、死代码消除、循环优化等典型优化环节，采用 spike 模拟器和高精度计时工具，对编译器生成代码的实际性能进行量化评估。由于 spike 模拟器本身存在一定精度限制，所有性能数据均为每类用例下每个测试用例重复运行 1000 次后的总耗时，统计结果更能反映整体趋势。

如图 4.3.1 所示，ToyC 编译器在各类优化场景下的表现略逊色于 GCC -O2。整体来看，ToyC 编译器能够较好地实现主要优化功能，验证了优化环节的有效性和自动化设计的合理性。

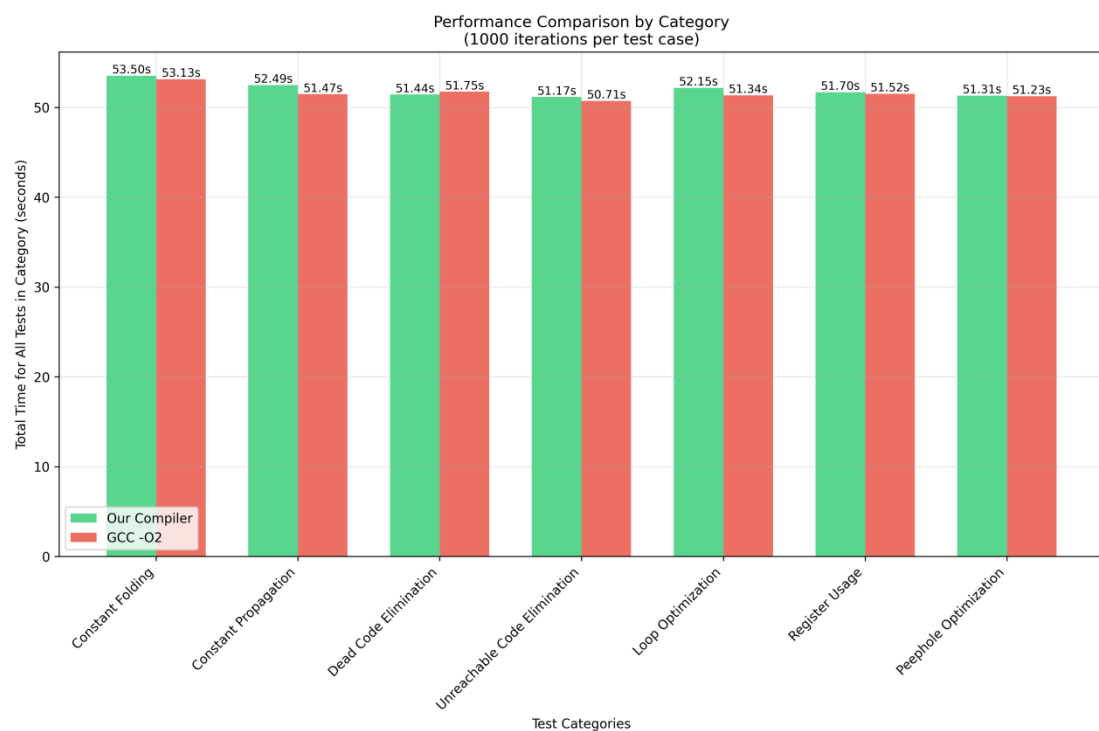


图 4.3.1 优化效果对比分析图

5 总结与展望

5.1 工作总结

本项目成功设计并实现了一个功能完备的优化编译器，它遵循现代三段式架构，可将一种类 C 的源语言（ToyC）编译为可执行的 RISC-V（RV64I）汇编代码。整个编译流水线始于前端，我们借助 `ocamllex` 与 `ocamlyacc` 工具，将源代码文本流转化为一个严格定义的抽象语法树（AST）。这个 AST 不仅捕获了源程序的句法结构，也为后续所有分析与变换提供了统一的入口。

编译器的核心位于中端，其设计的精髓在于引入了一套基于控制流图（CFG）的自定义中间表示（IR）。我们并未止步于简单的三地址码，而是实现了完整的静态单赋值（SSA）形式的转换。该过程严格遵循 Cytron 算法，通过不动点迭代精确计算出函数的支配树与支配边界，从而为 Φ 函数的放置与变量的系统性重命名提供了理论依据。在 SSA 形式之上，我们构建了一系列模块化的优化遍，包括迭代式的常量传播与折叠、基于 CFG 遍历的不可达代码消除、基于活变量分析的死代码消除，以及一个为小型、非递归函数设计的保守函数内联器。这些优化协同工作，系统性地提升了代码质量。当所有优化完成后，我们再通过 SSA 解构过程，将代码还原为可直接由后端处理的常规形式。

在后端，我们致力于解决将无限虚拟寄存器映射到有限物理寄存器这一核心矛盾。为此，我们完整地实现了一套基于图着色的全局寄存器分配算法。该算法的流程极为严谨：它首先通过精确的活跃性分析来构建干涉图，此图将变量生命周期的冲突关系模型化；随后，一个基于 Briggs 风格的启发式着色算法，通过“简化-合并-冻结-溢出”的迭代循环，为变量分配物理寄存器。我们特别为溢出（Spilling）设计了稳健的回退机制，当着色失败时，编译器会自动重写程序，引入必要的内存读写指令，并重新发起分配流程，直至成功。寄存器分配完成后，最终的指令生成模块负责将 IR 转化为 RISC-V 指令，并在此过程中审慎地处理函数调用规约，构建栈帧以管理本地变量与被保存的寄存器。作为最后的点睛之笔，一个窥孔优化遍会对生成的汇编代码进行扫描，以消除局部冗余，提升最终代码的微观效率。

5.2 不足与展望

尽管本项目已构建起一个完整的编译框架，但其深度与广度仍有拓展空间。在优化层面，当前的优化遍多为基础的标量优化。一个明确的改进方向是引入更强大的循环优化。虽然我们已经具备了循环检测的初步能力，但并未实现如循环不变量外提(LICM)或循环展开等能显著提升循环密集型程序性能的经典优化。此外，全局公共子表达式消除(GCSE)的缺失，也使得编译器在消除跨基本块的冗余计算方面有所欠缺。

在后端技术上，我们的图着色寄存器分配器虽然功能完备，但其启发式策略相对朴素。例如，合并(Coalescing)策略可以设计得更为激进，而溢出代价模型目前仅基于简单的启发式规则(如变量度数)，远未达到基于循环深度或使用频率等更精确的代价分析。指令选择阶段目前仍是较为直接的模板展开，未来可探索基于树或有向无环图(DAG)的模式匹配技术，以生成更优的指令序列。从语言支持的角度看，ToyC 作为一个教学语言，其特性是精简的。最自然的扩展方向是引入对指针与数组的支持。这不仅是对前端的挑战，更将对中端和后端的内存模型、别名分析(Alias Analysis)以及依赖分析提出全新的、更高的要求，是通往一个更实用编译器的必经之路。

最后，本项目的模块化设计为其未来的可移植性奠定了基础。一个富有挑战性的展望是将后端重定向至一个完全不同的指令集架构，如 ARM 或 x86。这将会充分检验我们前端、中端与后端之间接口设计的合理性与高内聚、低耦合的程度，也是对编译器工程化水平的一次终极考验。

6 小组成员和分工

本项目由刘鼎琨、郑慧琳、柯昀志、谢巧四人协作完成，各成员在编译器的不同核心模块承担了明确的职责，共同构建了从源代码到目标汇编的完整流水线。具体分工如图 6.1 所示。

姓名	分工
刘鼎琨	AST树转换，项目功能和数据结构设计
郑慧琳	常数、循环、死代码等多passes优化
谢巧	消除Phi，IR转Risc-V，及窥孔优化
柯昀志	设计自定义IR，及AST树转自定义IR

图 6.1:小组成员分工图

刘鼎琨同学负责项目的整体架构设计与前端模块的完整实现。在架构层面上，定义了编译器三段式（前端-中端-后端）的核心数据流与模块接口，确保了各部分能够独立开发并有效集成。在前端实现上，基于 `ocamllex` 工具编写了词法分析器（`lexer.mll`），将 `ToyC` 源代码字符流分解为词法单元序列；同时，利用 `ocamlyacc` 设计了语法规则（`parser.mly`），并定义了抽象语法树（AST）的核心数据结构（`ast.ml`）。该工作为整个编译流程提供了规范化、结构化的输入，是所有后续分析与优化的基础。

柯昀志同学专注于连接前端与中端优化的桥梁，核心工作是自定义中间表示（IR）的设计与 SSA 形式的构建。首先在 `myir.ml` 中设计了一套接近三地址码的底层 IR，该 IR 能够有效表示程序的运算、内存访问与控制流等多种操作。随后，实现了从 AST 到 IR 的转换逻辑（`ast2ir.ml`），将高层级的语法结构转化为基于控制流图（CFG）的线性指令序列。在此基础上，实现了 SSA 构建的核心分析算法（`ssa_builder.ml`），通过不动点迭代精确计算出支配树和支配边界，为后续优化遍的高效执行奠定了关键的数据结构基础。

郑慧琳同学负责中端一系列核心优化遍的开发与实现，旨在系统性地提升 IR 的代码质量。实现了循环分析与优化遍（`pass1_cycle.ml`），为识别和处理循环结构提供支持。同时还开发了迭代式的常量传播与折叠遍（`pass2_const.ml`），该优化可以在编译期直接计算出常量表达式的结果，并简化相关的分支判断。此外，还实现了不可达代码与死代码消除遍（`pass3_reachable.ml`），通过分析程序的控制流和数据流，精确地移除程序中永远不会被执行或不产生任何有效结果的

冗余代码，精简了程序结构。

谢巧同学负责从优化后的 IR 到最终 RISC-V 汇编代码生成的完整后端流水线。工作始于 SSA 解构 (`ssa_destruction.ml`)，通过消除 Φ 函数，将代码从 SSA 形式还原为常规的变量赋值形式。随后，还实现了基于图着色的全局寄存器分配器 (`ra.ml`)，通过活跃性分析、干涉图构建和启发式着色，为虚拟寄存器寻找最优的物理寄存器映射方案。接着，完成了 IR 到 RISC-V 的指令生成 (`ir2rv.ml`)，负责构建函数栈帧、翻译 IR 指令并处理函数调用规约。最后，实现了窥孔优化器 (`peephole.ml`)，对最终生成的汇编代码进行局部模式匹配和替换，进一步提升了代码效率。

致 谢

在本次编译器课程项目的学习与实践过程中，我们深刻体会到理论与工程结合的挑战与乐趣。首先，衷心感谢袁老师在整个课程期间的悉心指导和耐心培养。袁老师不仅在课堂上系统讲解了编译原理的核心知识，还在项目设计、技术选型、实现细节等方面给予了我们极大的帮助和鼓励，使我们能够顺利完成从理论到工程的转化。

同时，感谢钟学长、刘学姐在方案设计、技术难点攻关以及项目推进过程中给予的无私帮助和宝贵建议。他们的经验分享和耐心答疑为我们团队解决了许多实际问题，也让我们在遇到困难时始终能够保持信心和动力。

最后，感谢团队成员之间的积极协作与共同努力。每一次讨论、每一次代码迭代、每一次测试和优化，大家都能充分发挥各自的专长，互相支持、共同进步。正是这种团结协作的氛围，让我们在项目推进过程中不断成长，收获了宝贵的知识和友谊。

参考文献

- [1] Lattner, C., & Adve, V. (2004). LLVM: A compilation framework for lifelong program analysis & transformation. In *Proceedings of the 2004 International Symposium on Code Generation and Optimization (CGO)* (pp. 75-86). IEEE.
- [2] Cytron, R., Ferrante, J., Rosen, B. K., Wegman, M. N., & Zadeck, F. K. (1991). Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems*, *13*(4), 451-490.
- [3] Chaitin, G. J. (1982). Register allocation & spilling via graph coloring. *ACM SIGPLAN Notices*, *17*(6), 98-101.
- [4] Briggs, P., Cooper, K. D., & Torczon, L. (1994). Improvements to graph coloring register allocation. *ACM Transactions on Programming Languages and Systems*, *16*(3), 428-455.
- [5] Lattner, C. (2002). *LLVM: An infrastructure for multi-stage optimization*. Master's thesis, University of Illinois at Urbana-Champaign.
- [6] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). 编译原理(第2版). 机械工业出版社. [原书: Compilers: Principles, Techniques, and Tools (2nd Edition)]